Optimization Techniques
Electrical Computer and Engineering Dept.
Aristotle University of Thessaloniki
7th Semester - December 2025

Anastasia Loukeri
Student ID:10975
aloukeri@ece.auth.gr

General Problem Description

Goal: The objective is to find the minimum of a given multivariable function $f(x)$ with and without constraints and with utility of derivatives. More specifically, the search algorithms that will be studied are:

- **Steepest Descent Method**
- **Steepest Descent Method with Projection**

The algorithms we will use are based on the idea of iterative descent, based on which we start from a point $x_0 \in R^n$ and successively generate vectors $x_1, x_2, \ldots$ such that $f(x_{k+1}) < f(x_k), k = 1, 2, \ldots$. The objective function we will study is:

$$f(x_1, x_2) = x_1^2/3 + 3x_2^2$$

Mathematical Analysis

1. Gradient Calculation

To apply descent methods, we first calculate the gradient vector $\nabla f(x, y)$. The partial derivatives are:

$$\begin{aligned}\frac{\partial f}{\partial x_1} &= \frac{\partial}{\partial x_1}\left(x_1^2/3 + 3x_2^2\right)\\ &= 2/3x_1\end{aligned}$$

$$\begin{aligned}\frac{\partial f}{\partial x_2} &= \frac{\partial}{\partial x_2}\left(x_1^2/3 + 3x_2^2\right)\\ &= 6x_2\end{aligned}$$

Thus, the gradient is:

$$\nabla f(x_1, x_2) = \begin{pmatrix} 2/3x_1 \\ 6x_2 \end{pmatrix}$$

2. Critical Points

We find the critical points by setting $\nabla f(x_1, x_2) = \mathbf{0}$.

$$\begin{cases} 2/3x_1 = 0 & (1) \\ 6x_2 = 0 & (2) \end{cases}$$

From equation (1), we have only one critical point:

$$x_1 = 0 \quad \text{and} \quad x_2 = 0$$

3. Finding the Minimum

The function f is defined as convex over the convex domain X. The global minimum is located at the origin $(0, 0)$, where the objective function value is zero. This is substantiated by the first-order necessary condition; computing the gradient vector yields:

$$\nabla f(x_1, x_2) = \begin{bmatrix} \frac{2}{3}x_1 \\ 6x_2 \end{bmatrix}$$

Solving for the critical points by setting $\nabla f = \mathbf{0}$ identifies $(0, 0)$ as the unique stationary point. The Hessian matrix, $H(x) = \nabla^2 f$, is derived as follows:

$$H(x) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} \end{bmatrix} = \begin{bmatrix} \frac{2}{3} & 0 \\ 0 & 6 \end{bmatrix} = \begin{bmatrix} 0.66 & 0 \\ 0 & 6 \end{bmatrix}$$

The spectral analysis reveals the eigenvalues to be $\lambda_{\text{min}} = 0.66$ and $\lambda_{\text{max}} = 6$. The strict positivity of these eigenvalues demonstrates that the Hessian is positive definite. Consequently, the identified critical point is confirmed to be the strict global minimum of the function.

This analysis is visually confirmed by the plot of the function.

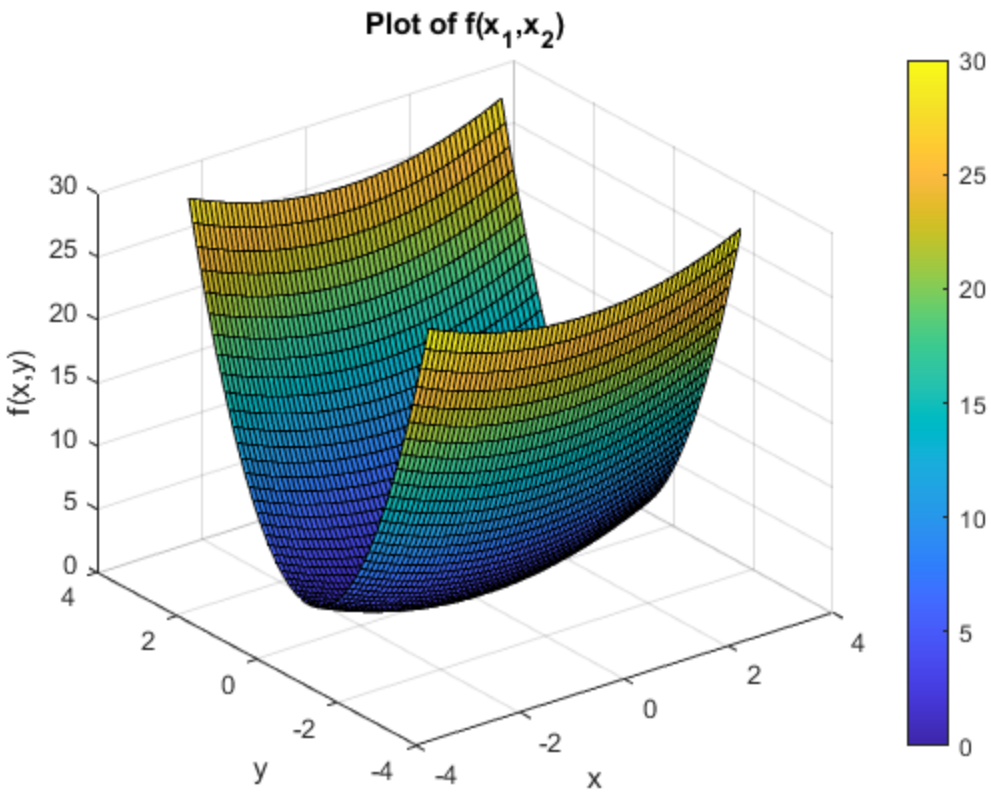


Figure 1: **3D Surface Plot of the Objective Function $f(x_1, x_2)$.**

Requirement 1 - Steepest Descent without constraints

In this method, to select the next point, we define $x_{k+1} = x_k + \gamma_k \cdot d_k$, where $d_k = -\nabla f(x_k)$. Our choice relies on the fact that the inner product $\nabla f^T(x_k) \cdot d_k$ is minimized when $d_k = -\frac{\nabla f(x_k)}{|\nabla f(x_k)|}$. With an appropriate choice of γ_k at each iteration, we can calculate the next point x_{k+1}.

For the implementation of the method, we followed Algorithm 5.2.1, page 121 of the book. Our deviation from this algorithm lies in the rule by which γ_k is selected, as in our implementation we have three different ways of making the selection, which we present below together with the results regarding the convergence (or non-convergence) of the algorithm.

Application to the Objective Function

We apply this method to our function:

$$f(x_1, x_2) = x_1^2/3 + 3x_2^2$$

First, recall the gradient vector derived in the previous section:

$$\nabla f(x_1, x_2) = \begin{bmatrix} \frac{2}{3}x_1 \\ 6x_2 \end{bmatrix}$$

We begin by establishing the theoretical bounds for the constant step size γ_k that guarantee the convergence of the method to the minimum. The iterative update rule is defined as: $x_{k+1} = x_k - \gamma_k \nabla f(x_k)$.
For the method to converge, the contraction condition must hold at every iteration:

$$\left|\frac{x_{k+1}}{x_k}\right| < 1 \Leftrightarrow \left|\frac{x_k - \gamma_k \nabla f(x_k)}{x_k}\right| < 1$$

Substituting the gradient formula derived in the previous analysis yields the following inequalities for each component:

$$\begin{matrix} \left|1 - \frac{2}{3}\gamma_k\right| < 1 \\ |1 - 6\gamma_k| < 1 \end{matrix} \quad \Longleftrightarrow \quad \begin{bmatrix} 0 < \gamma_k < 3 \\ 0 < \gamma_k < \dfrac{1}{3} \approx 0.33 \end{bmatrix}$$

It follows that for simultaneous convergence in both components with a constant step size γ_k, the condition $\gamma_k \in (0, 0.3\bar{3})$ must be satisfied.

Furthermore, solving the recurrence relations to determine the closed-form expressions for $x_{1,k}$ and $x_{2,k}$, we obtain:

$$\begin{bmatrix} x_{1,k} \\ x_{2,k} \end{bmatrix} = \begin{bmatrix} x_{1,0} \cdot (1 - 2/3 \cdot \gamma)^k \\ x_{2,0} \cdot (1 - 6 \cdot \gamma)^k \end{bmatrix}$$

These formulas further corroborate that for both components to converge, the condition $\gamma_k \in (0, 0.3\bar{3})$ is required. Additionally, these relationships elucidate the rate at which each component converges to zero, which is governed by the proximity of the respective coefficient (the term raised to the k-th power) to zero.

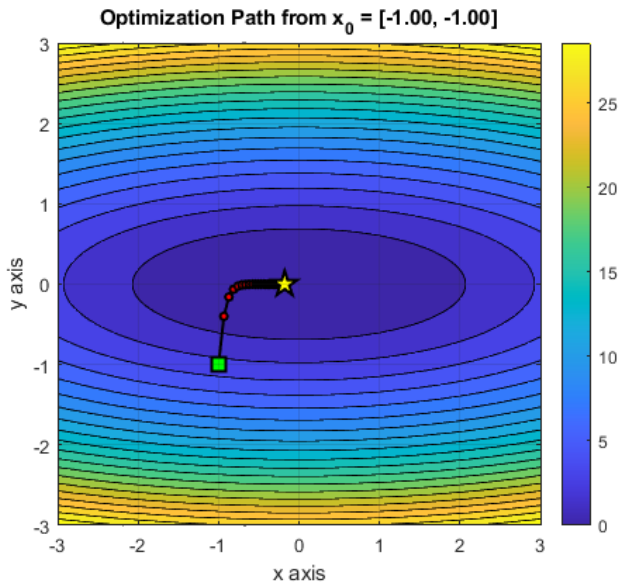
From the component-wise analysis above, we observe that the method (provided the convergence condition is met) will converge regardless of the initial starting point x_0. For the experimental phase, we select the initial point $(-1, -1)$ and execute the algorithm across various values of γ_k.

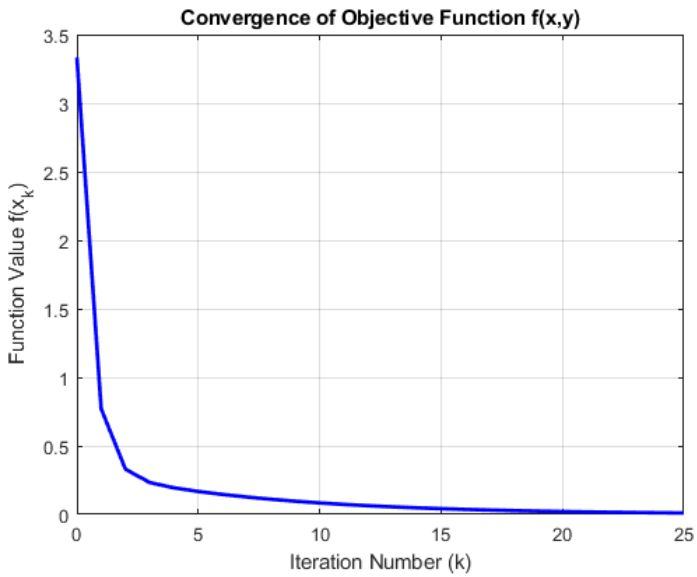
Case 1: $\gamma_k = 0.1$

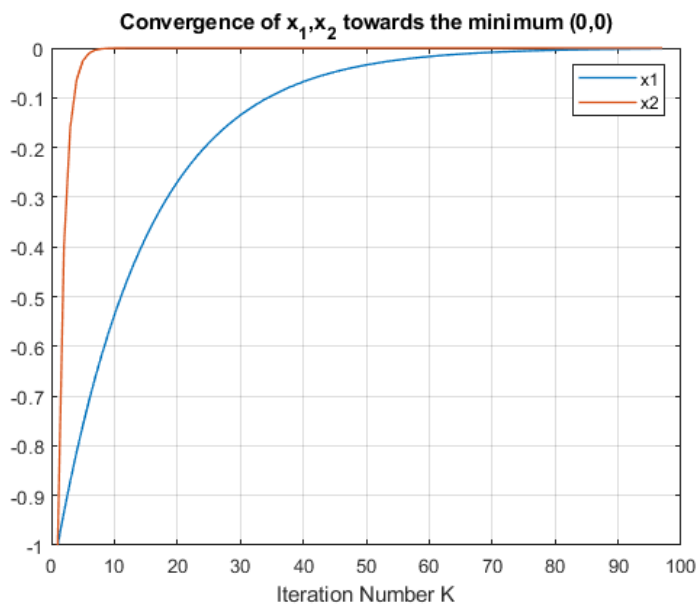
Substituting $x_1 = -1, x_2 = -1$ into the gradient:

$$\nabla f(-1,-1) = \begin{pmatrix} -2/3 \\ -6 \end{pmatrix}$$

Result: The algorithm will move in the negative x direction (towards the global minimum at $x_1 = 0$) and the negative y direction (towards $x_2 = 0$). The gradient here points clearly towards the minimum. The path is smooth and curves directly into the center of the blue region.The function value drops very quickly in the first few iterations (steep slope) and then settles at the minimum value of 0. This is a "monotonic decrease" meaning it gets better at every single step.







By solving the linear recurrence relations, we determine the closed-form expressions for $x_{1,k}$ and $x_{2,k}$ as a function of the iteration count k:

$$\begin{bmatrix} x_{1,k} \\ x_{2,k} \end{bmatrix} = \begin{bmatrix} x_{1,0} \cdot \left(1 - \frac{2}{3}\gamma\right)^k \\ x_{2,0} \cdot (1 - 6\gamma)^k \end{bmatrix}$$

These formulas corroborate the necessity of the condition $\gamma_k \in (0, 0.3\bar{3})$. Furthermore, they elucidate the rate at which each component approaches zero. For a specific valid step size (e.g., $\gamma = 0.1$), the evolution is given by:

$$\begin{bmatrix} x_{1,k} \\ x_{2,k} \end{bmatrix} = \begin{bmatrix} x_{1,0} \cdot (0.9\bar{3})^k \\ x_{2,0} \cdot (0.4)^k \end{bmatrix}$$

Observation: A contraction coefficient with a smaller magnitude (closer to 0) induces faster convergence, as it provides a more significant reduction of the error value per iteration. Consequently, we observe that the component $x_{2,k}$ converges significantly faster than $x_{1,k}$ because $|0.4| < |0.9\bar{3}|$.

Case 2: $\gamma_k = 0.3$

Similarly, the algorithm converges as the step size remains within the valid range $\gamma_k \in (0, 0.3\bar{3})$. In this scenario, both components share the same magnitude of convergence rate ($|0.8|$), but exhibit qualitatively different behaviors:

$$\begin{bmatrix} x_{1,k} \\ x_{2,k} \end{bmatrix} = \begin{bmatrix} x_{1,0} \cdot (0.8)^k \\ x_{2,0} \cdot (-0.8)^k \end{bmatrix}$$

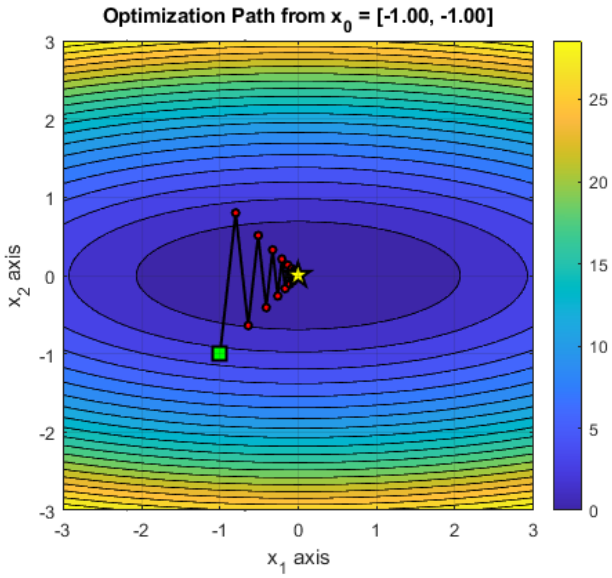
- **Component x_1:** The coefficient is positive (0.8), resulting in monotonic decay.
- **Component x_2:** The coefficient is negative (-0.8). This causes the value to oscillate (change sign) at every iteration while its magnitude dampens towards zero.

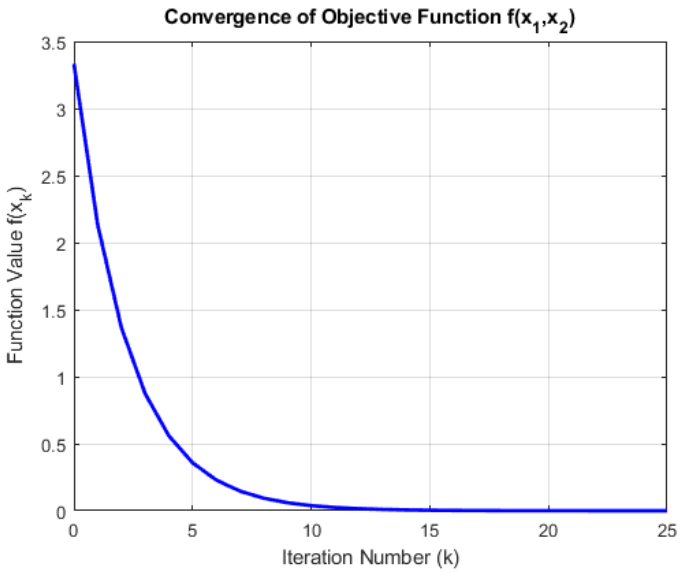
We observe that this step size ($\gamma = 0.3$) requires fewer than half the iterations compared to the previous case ($\gamma = 0.1$) to reach the same error tolerance. This is because the convergence rate is higher (the magnitude 0.8 is smaller than 0.93), even though the path is oscillatory. The contour plot illustrates the "Zig-Zag" phenomenon characteristic of Steepest Descent on ill-conditioned problems.

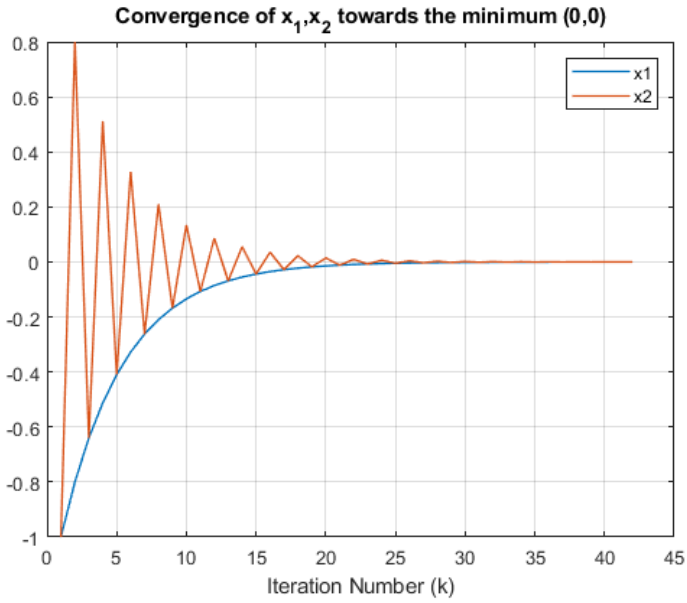
The optimization path starts at $[-1, -1]$. The step size is "aggressive" for the steep x_2 direction, causing the algorithm to overshoot the valley floor repeatedly. This corresponds to the oscillating term $(-0.8)^k$ derived in this second case. Conversely, the step size is conservative for the flat x_1 direction, resulting in a slow, steady approach.

The **Convergence of coordinates** plot visualizes the decay of the coordinate values over iterations k.

- **Blue Line (x_1):** Exhibits *monotonic convergence.* The curve is smooth and approaches zero from one side. This confirms a positive recurrence coefficient.
- **Red Line (x_2):** Exhibits *damped oscillation.* The value alternates between positive and negative, forming an envelope that decays to zero. This confirms a negative recurrence coefficient ($-1 < 1 - 6\gamma < 0$), which occurs when the step size γ is large enough to overshoot the minimum in the stiff direction (x_2).







Case 3: γ_k=3

The selection of a step size $\gamma = 3$ drastically violates the stability condition derived in the previous section, which required $\gamma \in (0, 0.3\bar{3})$. We can quantify this instability by substituting $\gamma = 3$ into the closed-form recurrence relation:

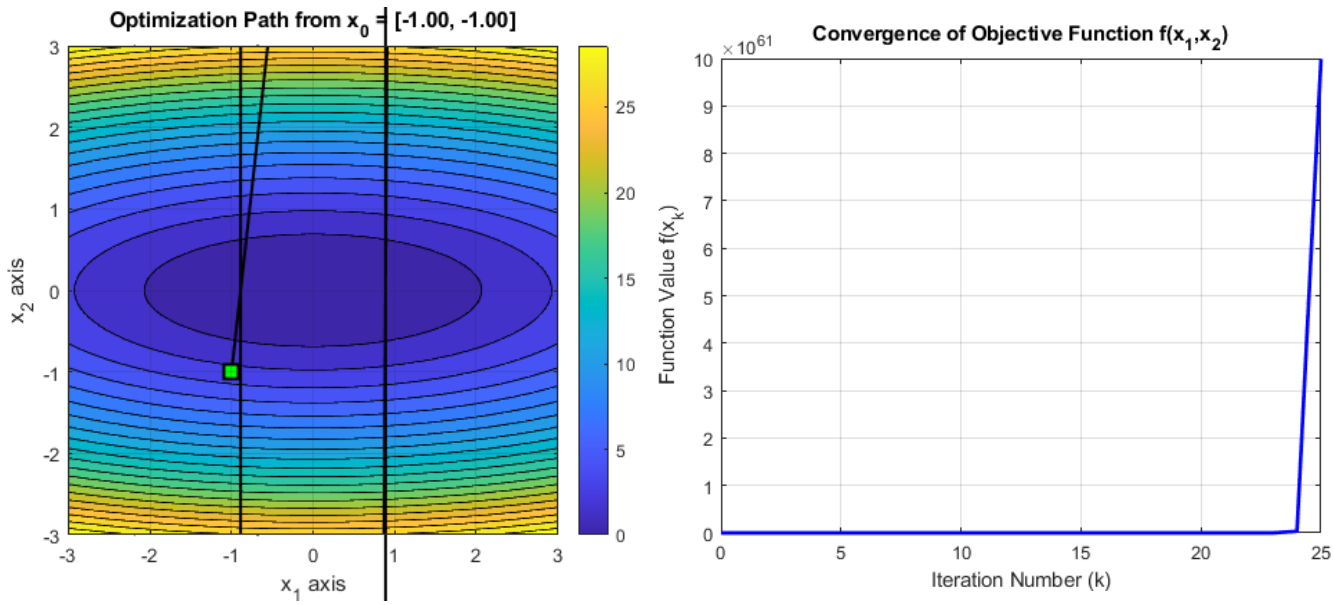
$$\begin{bmatrix} x_{1,k} \\ x_{2,k} \end{bmatrix} = \begin{bmatrix} x_{1,0} \cdot \left(1 - \frac{2}{3} \cdot 3\right)^k \\ x_{2,0} \cdot (1 - 6 \cdot 3)^k \end{bmatrix} = \begin{bmatrix} x_{1,0} \cdot (-1)^k \\ x_{2,0} \cdot (-17)^k \end{bmatrix}$$

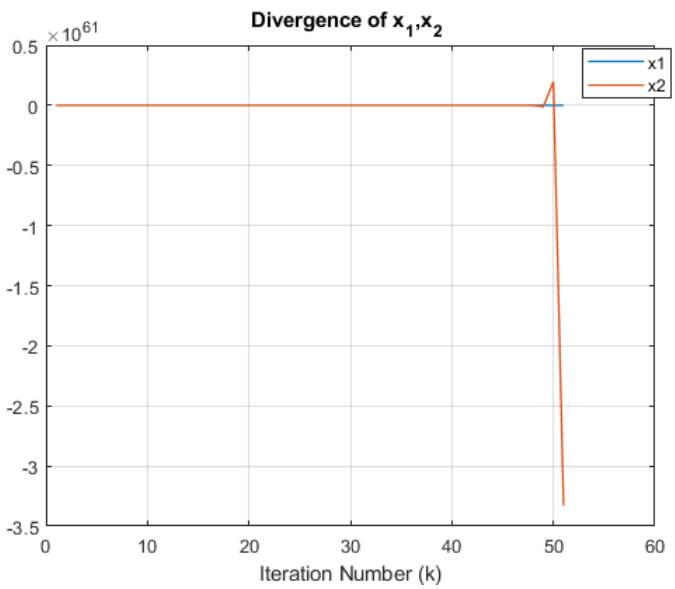
This formulation reveals the behavior of the system:

- **Component x_1:** The coefficient -1 implies undamped oscillation. The value simply flips signs at every iteration, never converging to the origin.
- **Component x_2:** The coefficient -17 indicates that the distance from the minimum increases by a factor of 17 at every step. This is an unbounded geometric progression

leading to immediate numerical overflow.

The provided plots confirm this analytical prediction:
They show the objective function value and the x_2 coordinate reaching magnitudes of 10^{61} within roughly 50 iterations. This is consistent with the exponential growth term $(-17)^k$. On the contour plot, the trajectory exits the visible domain immediately. The path appears as near-vertical lines because the divergence in the x_2 axis (steep curvature) dwarfs the oscillatory behavior in the x_1 axis.





Case 4: $\gamma_k = 5$

In this final test, we increased the step size to $\gamma = 5$. Based on our initial stability analysis, we expected this to cause a complete failure of the algorithm. Recall that for the method to work, the step size needs to be within specific bounds for each direction:

- For the x_1 direction, we needed $\gamma < 3$.
- For the x_2 direction, we needed $\gamma < 0.33$.

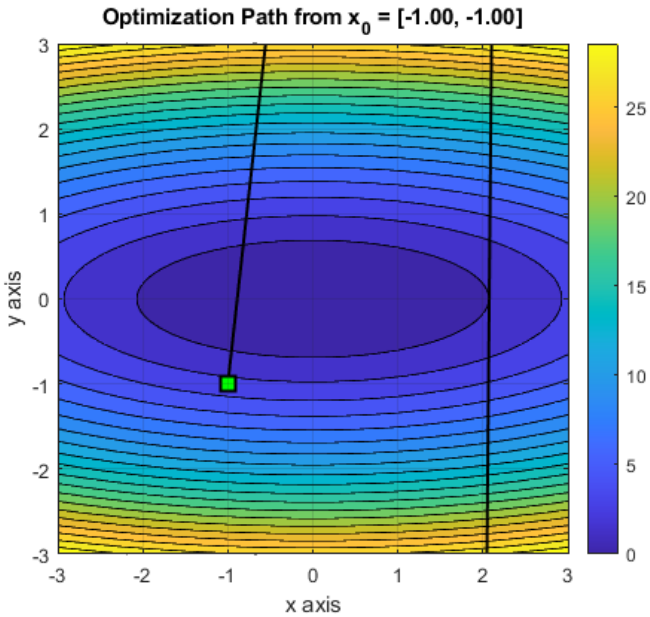
With $\gamma = 5$, we are violating **both** limits. This is different from the previous case ($\gamma = 3$), where x_1 was just on the edge of stability. Now, both directions are theoretically unstable. We can see exactly how bad the divergence will be by plugging $\gamma = 5$ into our update formulas:

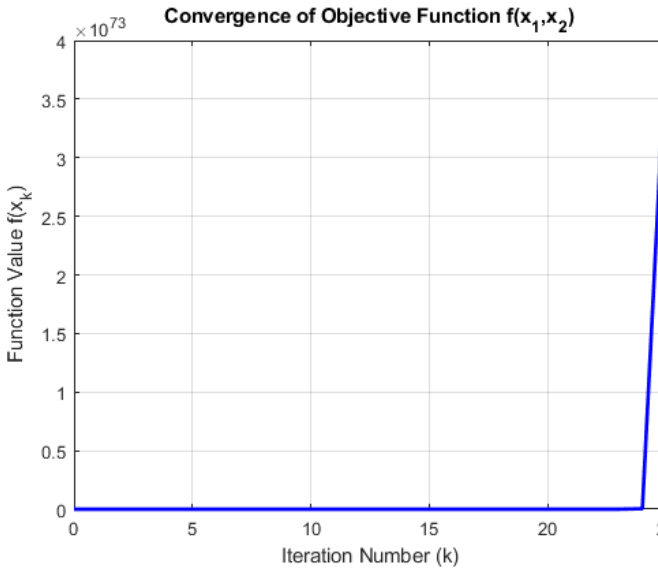
$$\begin{bmatrix} x_{1,k} \\ x_{2,k} \end{bmatrix} = \begin{bmatrix} x_{1,0} \cdot \left(1 - \frac{2}{3} \cdot 5\right)^k \\ x_{2,0} \cdot (1 - 6 \cdot 5)^k \end{bmatrix} = \begin{bmatrix} x_{1,0} \cdot (-2.33)^k \\ x_{2,0} \cdot (-29)^k \end{bmatrix}$$

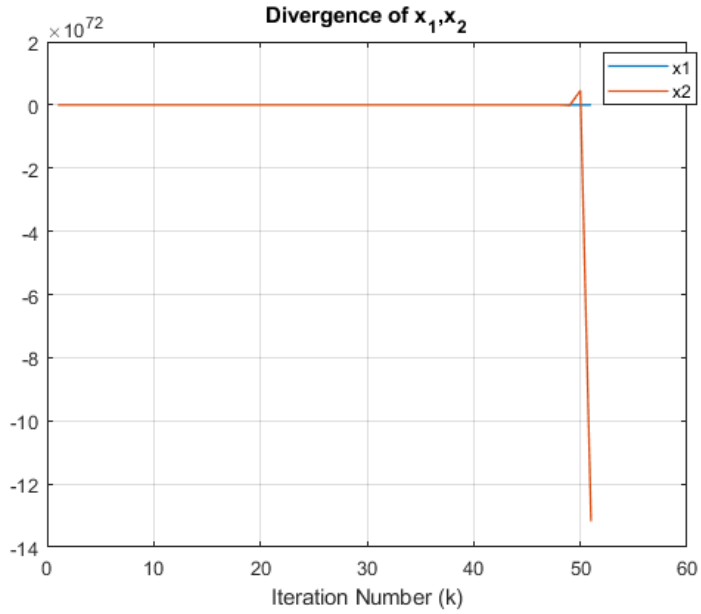
The equations above show us two important things happening at the same time:

- x_1 **is now unstable:** The multiplier is -2.33. Since this is greater than 1, the algorithm is now pushing us away from the minimum.
- x_2 **is exploding:** The multiplier is -29. This means at every single step, our error gets multiplied by 29 and flips signs. This is a massive geometric growth.

The plots from our simulation confirm that the system diverges instantly.
We have set an iteration limit for the algorithm to avoid an infinite loop. However, the plots do not display all values for every iteration because, for a very large number of iterations, MATLAB considers the numerically large result as infinite (or NaN).







- **Divergence of Coordinates:** In the component plot, we can see the x_2 line (red) shooting straight up. Within about 50 iterations, the values reach a magnitude of 10^{72}. The x_1 divergence is technically happening too, but it's so small compared to 10^{72} that it just looks flat on this scale.
- **Objective Function:** The graph of the function value shows the same vertical explosion. Because our function $f(x)$ squares the coordinates, the function value gets even bigger, reaching values around 10^{73}.
- **The Path:** On the contour plot, the trajectory doesn't even stay in the frame. It starts at $(-1, -1)$ and immediately shoots off the visible chart because the steps are way too aggressive for the curvature of the function.

Requirement 2 - Steepest Descent with Projection

1. Definition of the Constraint Set

As established in the introduction, the optimization is now bound by the constraint set X, defined as:

$$X = \{[x_1, x_2]^T \in R^2 : -10 \leq x_1 \leq 5, \quad -8 \leq x_2 \leq 12\} \subset R^2$$

This set forms a rectangle in the 2D plane. Since all rectangles are intersections of half-spaces, X is a **convex set**. Furthermore, the unconstrained global minimum of our objective function, $(0, 0)$, lies strictly within the interior of this set ($0 \in [-10, 5]$ and $0 \in [-8, 12]$). This allows us to utilize **Algorithm 6.1.1 (Algorithms with Projection)**, effectively using the Steepest Descent method modified with a projection step.

2. The Projected Update Rule

Analytically, the update rule consists of two stages: taking a standard gradient step, and then projecting that result back onto the set X. Let $\bar{x}_k$ be the tentative point before projection:

$$\bar{x}_k = x_k - s_k \nabla f(x_k)$$

The actual next iterate x_{k+1} is defined as:

$$x_{k+1} = Pr_X\{\bar{x}_k\} = Pr_X\{x_k - s_k \nabla f(x_k)\}$$

Where Pr_X is the Euclidean projection operator onto set X. If the tentative point falls **inside** X, the projection operation does nothing, and the method behaves exactly like standard Steepest Descent:

$$x_{k+1} = x_k - s_k \nabla f(x_k) \quad \text{if } (x_k - s_k \nabla f(x_k)) \in X$$

3. The Projection Mechanism

The projection of a point onto a rectangular set is equivalent to "clamping" each coordinate to the boundaries. The explicit mathematical formulation is:
For the x_1 component:

$$\bar{x}_{1,k} = \begin{cases} -10 & \text{if } x_{1,k}(1 - \frac{2}{3}s_k) < -10 \\ x_{1,k}(1 - \frac{2}{3}s_k) & \text{if } -10 \leq x_{1,k}(1 - \frac{2}{3}s_k) \leq 5 \\ 5 & \text{if } x_{1,k}(1 - \frac{2}{3}s_k) > 5 \end{cases}$$

For the x_2 component:

$$\bar{x}_{2,k} = \begin{cases} -8 & \text{if } x_{2,k}(1 - 6s_k) < -8 \\ x_{2,k}(1 - 6s_k) & \text{if } -8 \leq x_{2,k}(1 - 6s_k) \leq 12 \\ 12 & \text{if } x_{2,k}(1 - 6s_k) > 12 \end{cases}$$

4. Stability and Convergence Implications

The projection mechanism acts as a "safety net." In unconstrained cases, large step sizes led to unbounded divergence. However, with projection:

1. **Boundedness:** The algorithm forces the solution to remain within X. Even if the step size is large, the point simply "hits the wall" rather than escaping to infinity.

2. **Convergence:** For the algorithm to settle at the global minimum $(0, 0)$ without oscillating, the step size must satisfy the standard contraction mapping conditions:

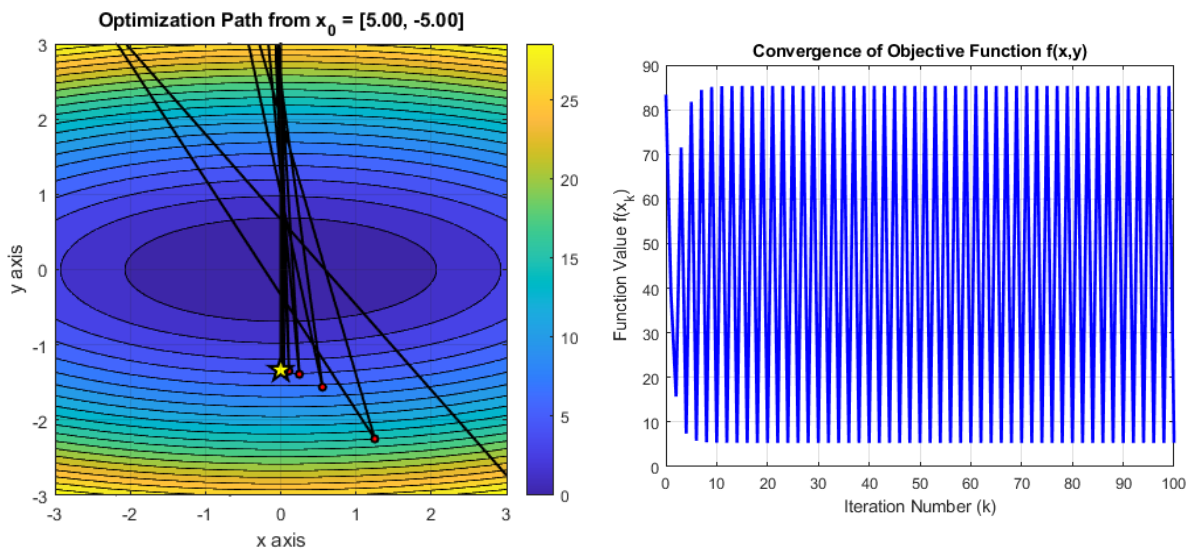
$$\left| \frac{x_{1,k+1}}{x_{1,k}} \right| < 1 \implies 0 < s_k \gamma_k < 3$$

$$\left| \frac{x_{2,k+1}}{x_{2,k}} \right| < 1 \implies 0 < s_k \gamma_k < \frac{1}{3}$$

In summary, the projection guarantees the iterates remain bounded, but the step size γ must still be small enough (< 0.33) to stop the particle from bouncing perpetually off the walls and actually converge to the center.

Case 1: Initial Point $(5, -5)$ and s_k=5, γ_k=0.5

To analyze the behavior of the Projected Gradient Descent algorithm with the scaling factor $s_k = 5$ and step size $\gamma_k = 0.5$, we first determine the "effective step size" of the iteration, which is the product $\eta = s_k \cdot \gamma_k = 2.5$. Comparing this value against the stability bounds derived in our earlier theoretical analysis reveals a critical discrepancy between the two dimensions. While the effective step size of 2.5 satisfies the stability condition for the x_1 component (which requires $\eta < 3$), it drastically exceeds the stability limit for the x_2 component, which requires $\eta < 0.33$. Consequently, while the algorithm is theoretically capable of converging in the flatter x_1 direction, the aggressive step size ensures that the x_2 component will never settle at the minimum. Consequently, the function value $f(x)$ oscillates between the two levels corresponding to the boundaries (7,85) and never settles to the minimum.



Case 2: Initial Point $(-5, 10)$ and s_k=15, $\gamma_k = 0.1$

Based on the provided parameters and the resulting plots, we present the scientific analysis of the system's behavior.

Calculation of Step Size

To interpret the results, we first calculate the effective step size η, which drives the iteration update:

$$\eta = s_k \cdot \gamma_k = 15 \cdot 0.1 = 1.5$$

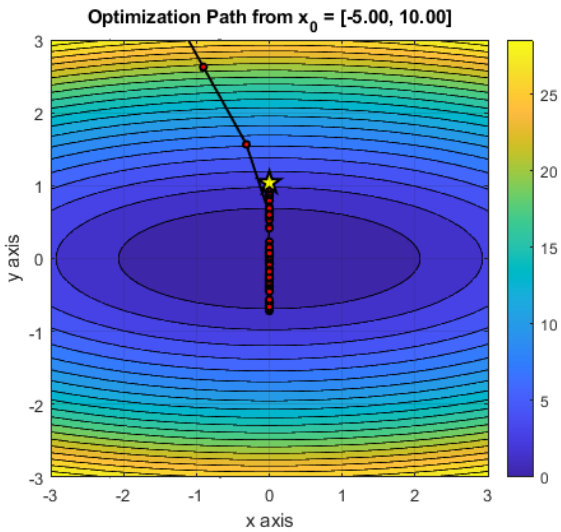
We compare this value against the stability criteria established in the theoretical analysis: For x_1 stability requires $\eta < 3$. Since $1.5 < 3$, this component is **stable**.For x_2 stability requires $\eta < 0.33$. Since $1.5 \gg 0.33$, this component is **unstable**.

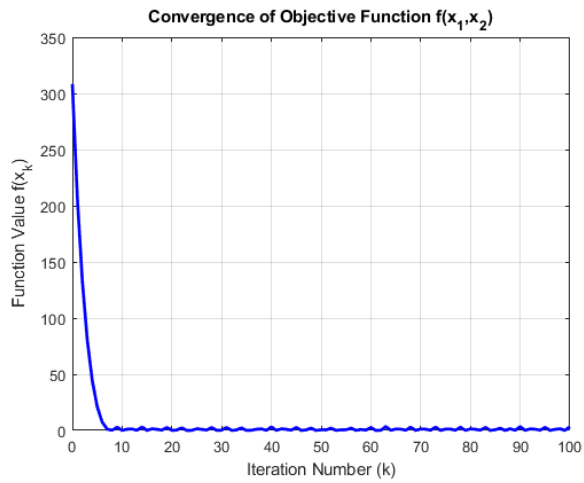
Behavior of Coordinates

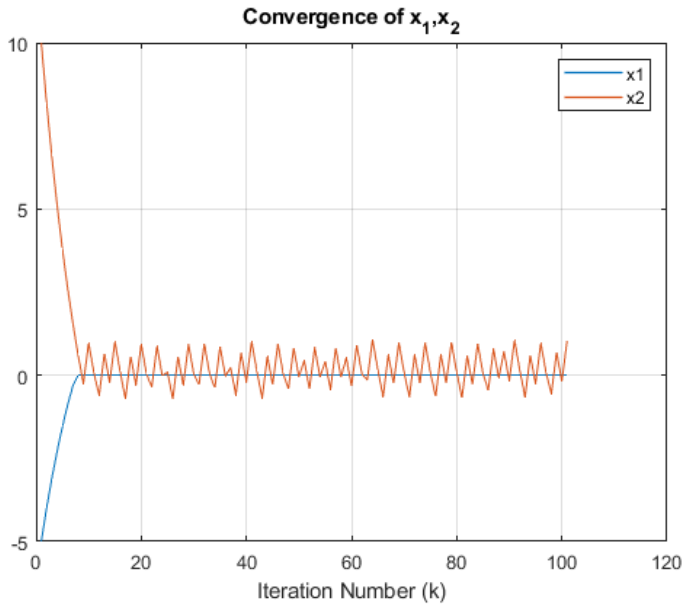
For the x_1 component the update rule is $x_{1,k+1} = x_{1,k}(1 - \frac{2}{3}\eta)$. Substituting $\eta = 1.5$:

$$x_{1,k+1} = x_{1,k}\left(1 - \frac{2}{3} \cdot 1.5\right) = x_{1,k}(1 - 1) = 0$$

The recurrence coefficient becomes exactly zero. This implies that the algorithm finds the exact coordinate of the minimum ($x_1 = 0$) in a single step. This is visually confirmed in the coordinate plot, where the blue line (x_1) drops from -5 to 0 immediately and remains there. Conversely, the x_2 component is driven by the multiplier $(1 - 6\eta) = (1 - 9) = -8$. This indicates massive expansion and oscillation. Consequently, the x_2 component enters a limit cycle, bouncing permanently. This is depicted in the red line of the convergence plot, which oscillates rapidly after the initial drop.

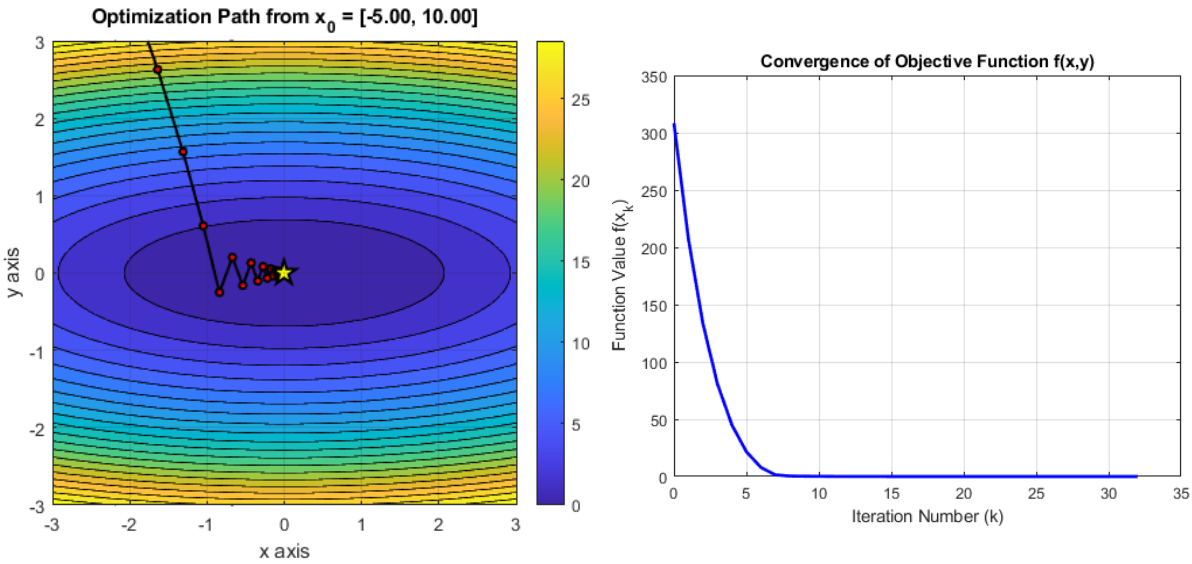


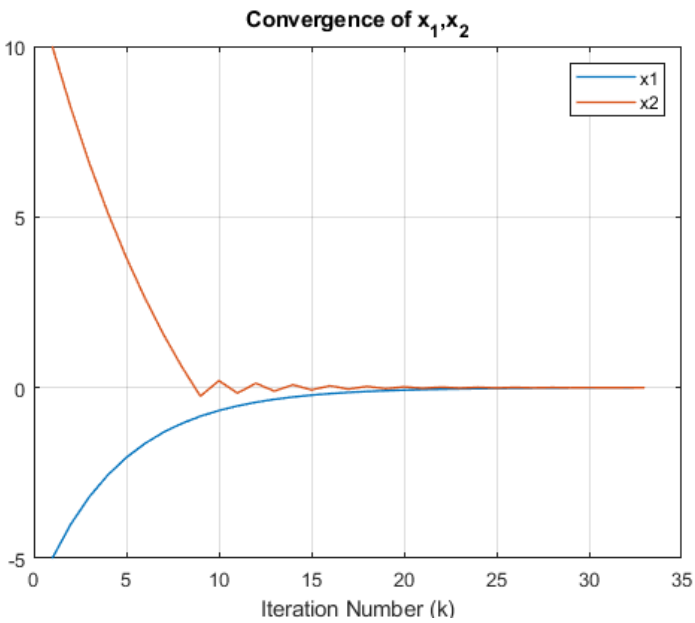




Convergence from $(-5, 10)$ with $s_k = 3$ and $\gamma_k = 0.1$

By selecting a scaling factor of $s_k = 3$, we successfully tune the effective step size to $\eta = 0.3$ (assuming the standard base step $\gamma_k = 0.1$), which falls precisely within the narrow stability window required by the system's stiffest component. As derived in our theoretical analysis, the critical threshold for convergence in the x_2 direction is 0.33; therefore, operating at 0.3 effectively maximizes the convergence speed without crossing the threshold into instability. This optimal tuning resolves the limit-cycle behaviors observed with higher parameters (such as $s_k = 5$ or $s_k = 15$), as the step magnitude is now sufficiently restrained to allow the trajectory to land within the valley of the objective function rather than overshooting between constraint boundaries, thereby ensuring asymptotic convergence to the global minimum $(0, 0)$.





Case 3: Initial Point $(8, -10)$ and $s_k = 0.1$, $\gamma_k = 0.2$

This time we see that the point is initially outside the region X, but with the first iteration we will obtain the projections of the components for $\bar{x}_{1,k}$ and $\bar{x}_{2,k}$ (as we showed in the theoretical analysis), therefore we will start moving in each iteration towards the region. When $\bar{x}_{1,k}$ and $\bar{x}_{2,k}$ are found inside the convex set, the relations for the next iterations that we derived will hold:

$$x_{1,k+1} = x_{1,k}\left(1 - \frac{2}{3}s_k\gamma_k\right), \quad x_{2,k+1} = x_{2,k}(1 - 6s_k\gamma_k).$$

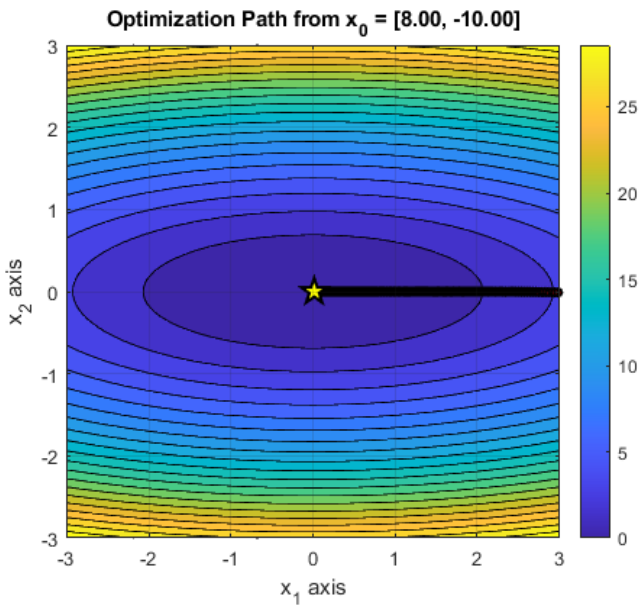
Therefore, convergence to $(0, 0)$ is ensured, given that $\gamma_k s_k = 0.02 < 0.33$. Substituting results in:

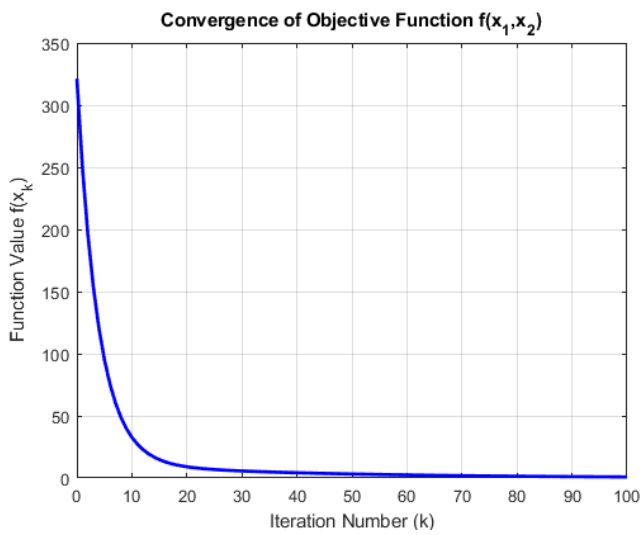
$$x_{1,k+1} = x_{1,k} \cdot 0.9866, \quad x_{2,k+1} = x_{2,k} \cdot 0.88.$$

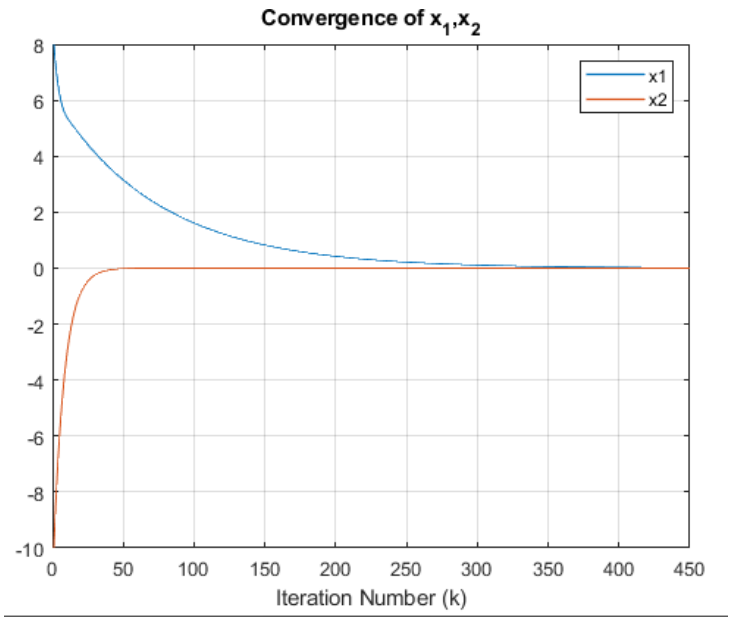
These relations show the gradual/slow reduction of $x_{1,k}$ and $x_{2,k}$ until they converge to zero. Also, given that $x_{2,k}$ has a smaller coefficient, as we also see in the corresponding diagram, it converges faster to 0.

The initial rapid drop in the functions' convergence plot corresponds to the elimination of the x_2 component. Once $x_2 \approx 0$, the remaining cost is dominated by x_1. Since x_1 decays very slowly (0.9866), the function value flattens out, indicating that the solver struggles to squeeze out the last bit of error.

These plots confirm that while the system is guaranteed to converge (as both coefficients are < 1), the convergence speed is non-uniform. The optimization effectively happens in two timescales: a fast transient phase where x_2 is resolved, followed by a slow steady-state phase dominated by the decay of x_1.






Convergence of x_1,x_2
8
6
4
2
0
-2
-4
-6
-8
-10
0
50
100
150
200
250
300
350
400
450
Iteration Number (k)
x1
x2